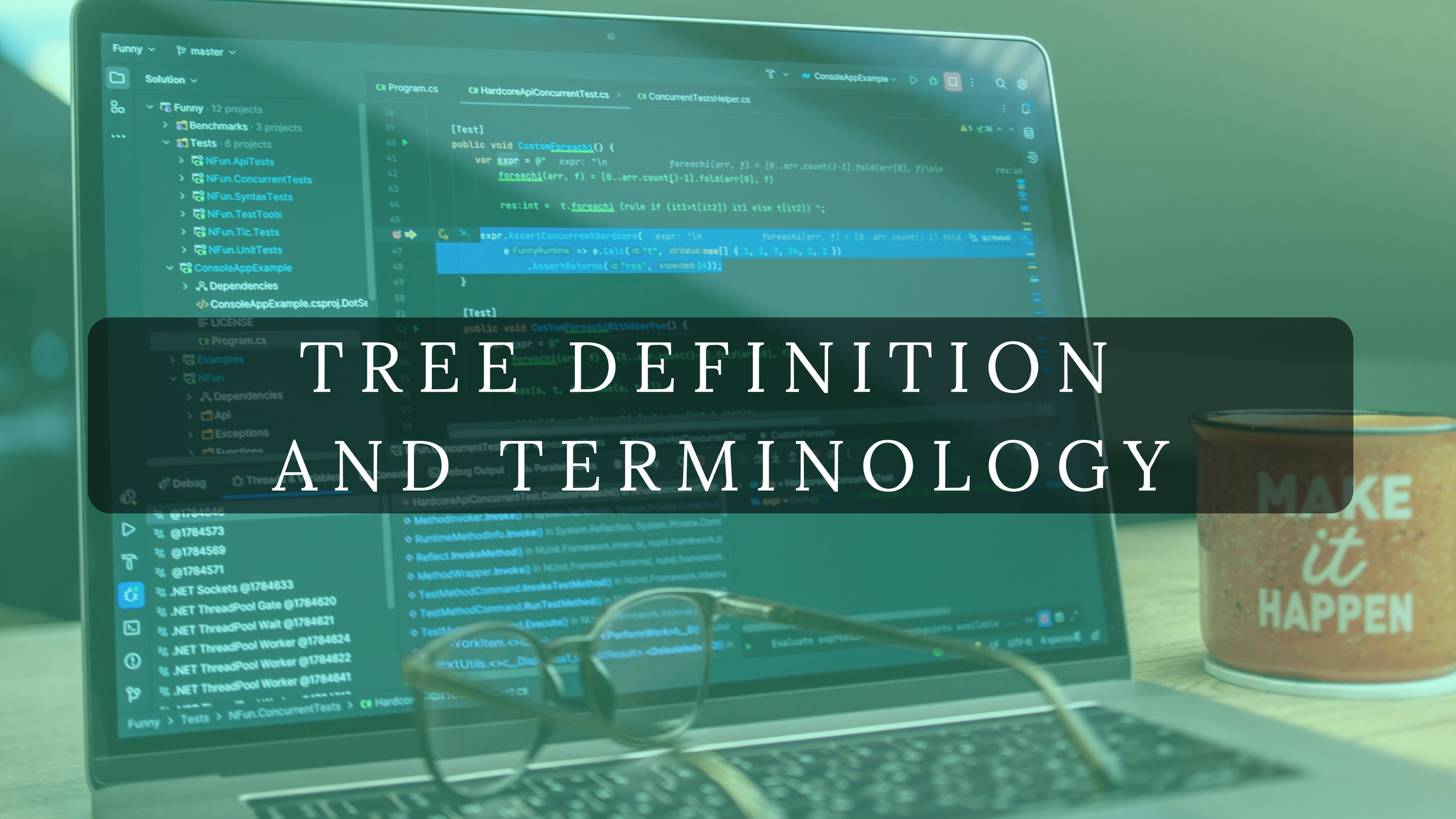
TREE THEORY

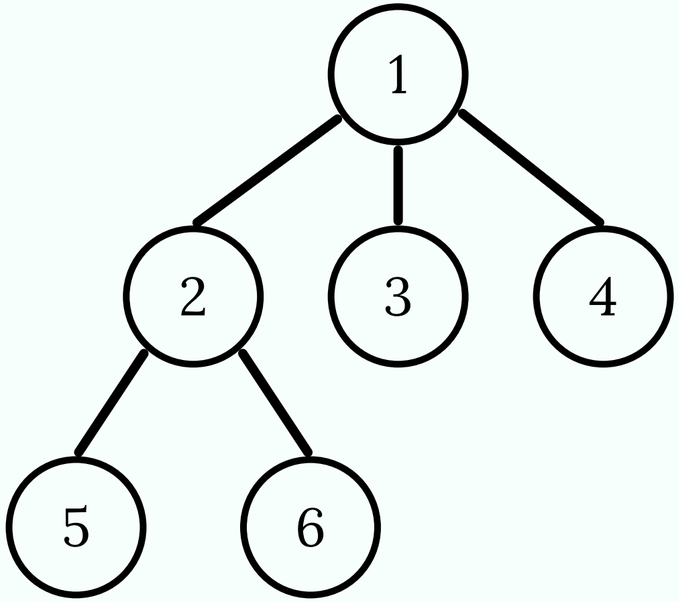
Military Institute of Science and Technology | 2026

OVERVIEW

- Tree Definition and Terminology
- Binary Trees
  - Types of Binary Trees
  - Binary Tree Properties
  - Binary Tree Representation
    - Linked Node Representation
    - Array Representation
- Tree Traversal Techniques


TREE DEFINITION
AND TERMINOLOGY

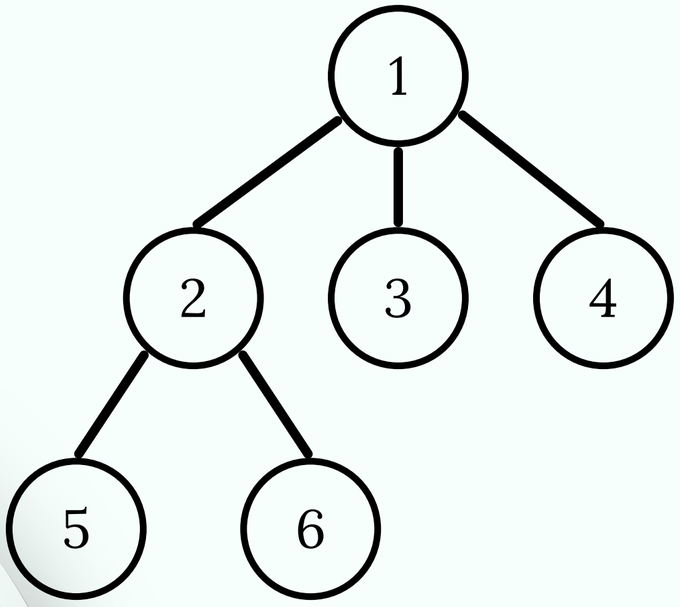
TREE DEFINITION



A tree is a special type of graph, which is acyclic and has exactly n-1 number of edges

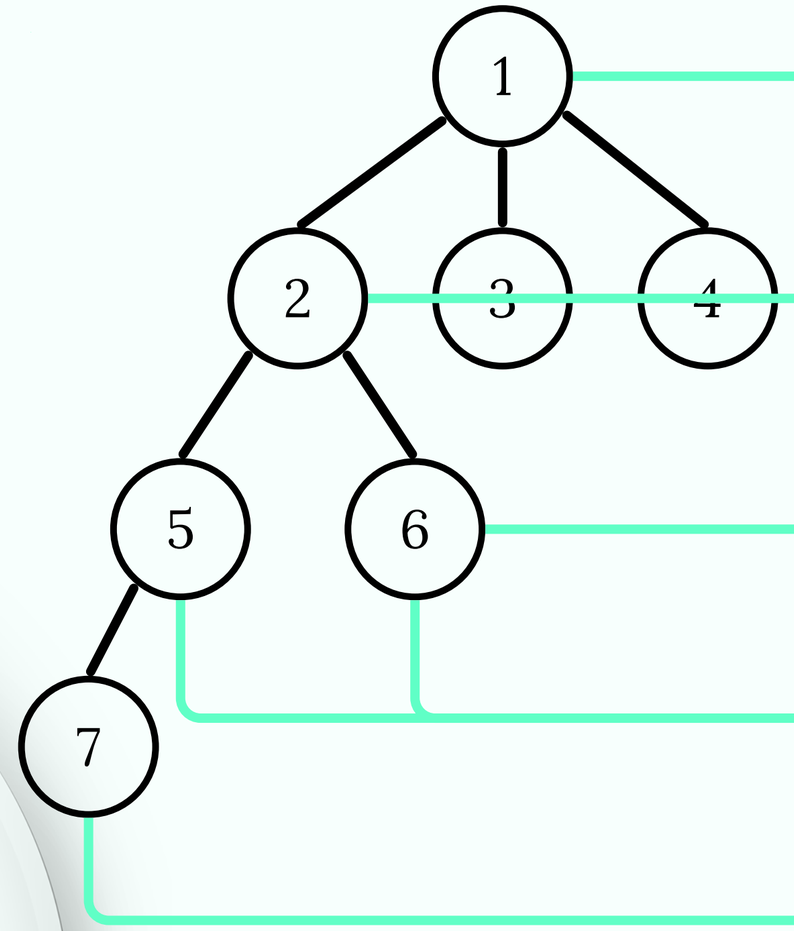
- A tree is basically a connected and acyclic graph.
- All trees are graphs, but all graphs are not trees

TREE PROPERTIES



- **Connectivity:** There are no disconnected nodes
- **Acyclic:** A tree has no loops or cycles
- **Number of Edges:** For a tree with **n** vertices, there are always **n-1** edges
- **Unique Paths:** There is exactly one unique path between any two nodes

TREE TERMINOLOGY



Root: The node without a parent. Every tree has exactly one root. (Here it is node **1**)

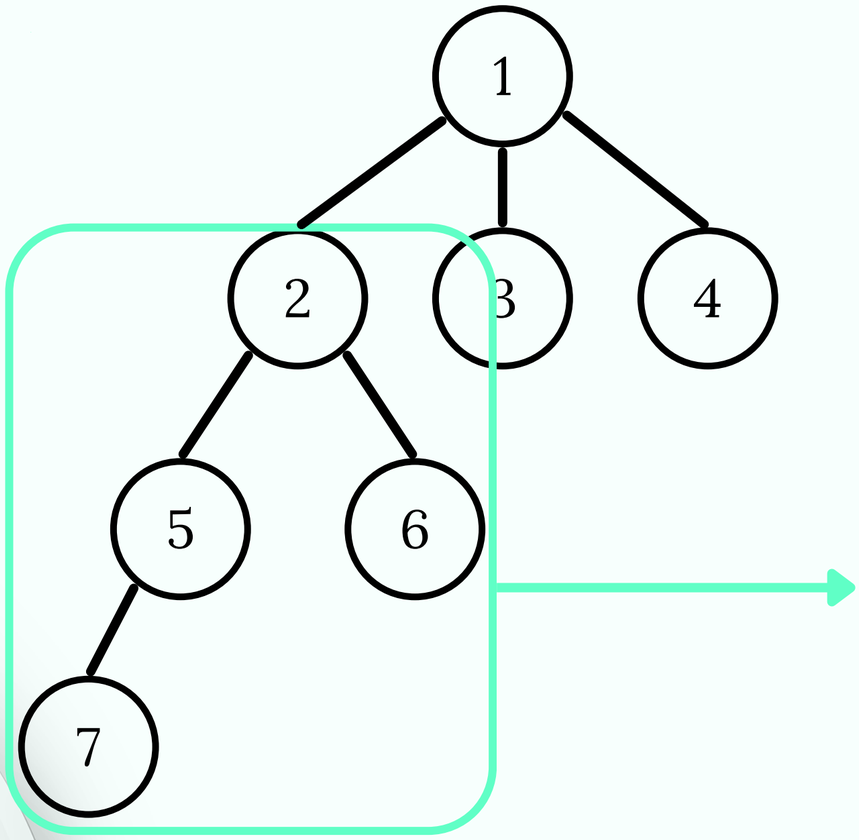
Parent: Any node that has one more nodes directly connected below it. (**2** is the parent of **5** and **6**)

Child: Any node directly connected below another node (its parent). (**5** and **6** are children of **2**)

Sibling: Nodes that share the same parent (**5** and **6** are siblings. **2**, **3** and **4** are also siblings)

Leaf Node: A node with no children (**7**, **6**, **3**, **4** are leaf nodes)

TREE TERMINOLOGY

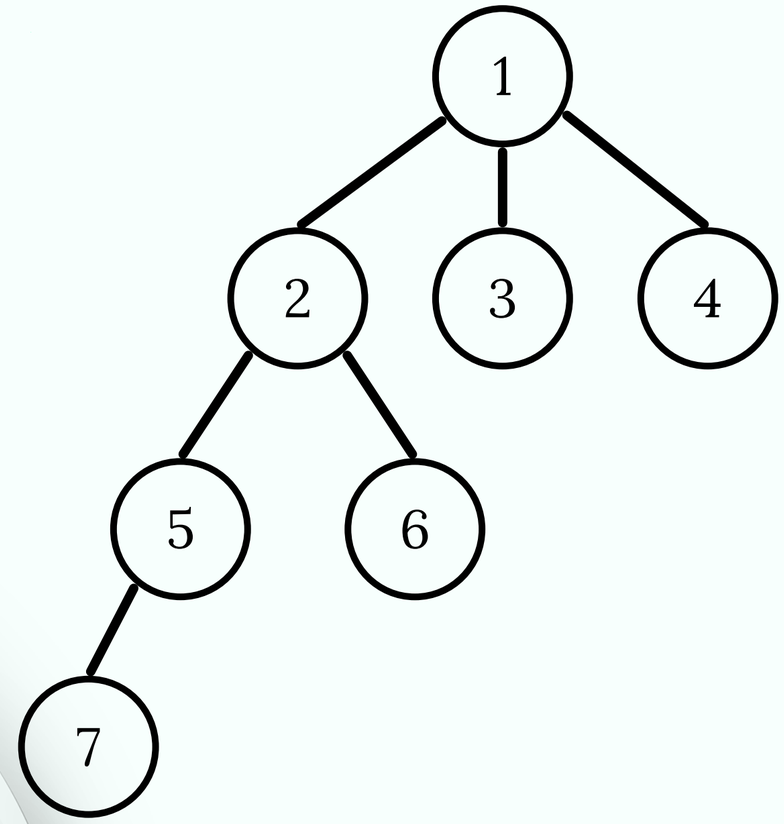


Ancestor: Any node on the path from a given node up to the root (**5**, **2**, **1** are ancestors of **7**)

Descendant: Any node reachable by moving downward from a given node (**5**, **6** and **7** are descendants of **2**)

Subtree: A portion of a tree. Here **2567** can be considered a subtree.

TREE TERMINOLOGY

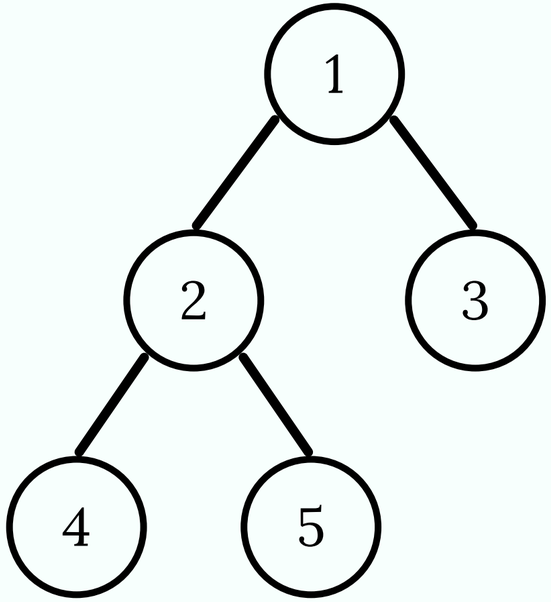


- **Height:** The maximum number of edges present in the path from a leaf node to a given node in its subtree.
  - Examples
    - Height of 5 = 1
    - Height of 1 = 3
- **Depth:** The number of edges present in the path from root to a given node.
  - Examples
    - Depth of 5 = 2
    - Depth of 1 = 0
- **Level:** same as Depth. (Some sources state level=depth+1)
  - Examples
    - Level of 5 = 2
    - Level of 1 = 0

BINARY TREES

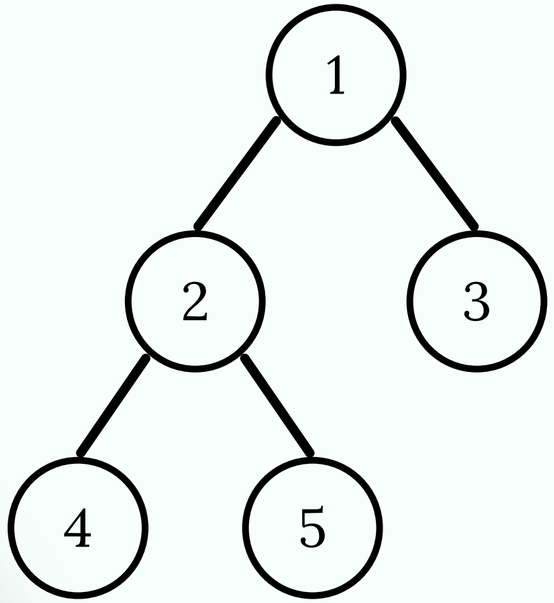


BINARY TREE DEFINITION



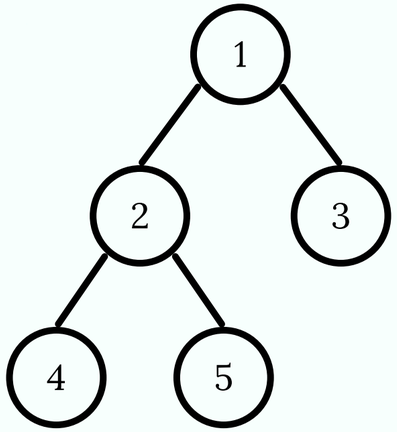
A binary tree is a tree in which each node has at most 2 child nodes.

BINARY TREE PROPERTIES

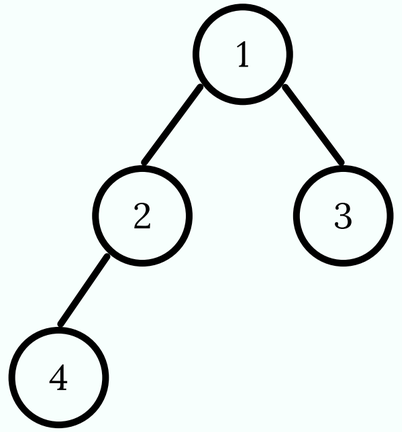


- **Child Nodes:** Each node will have at most 2 nodes, called **left** and **right**
- **Number of nodes:** A binary tree can have at most $\mathbf{2^L}$ nodes at level **L**.
- **Maximum number of nodes**
  - at depth d is $\mathbf{2^d}$
  - in a tree of height h is $\mathbf{2^{(h+1)} - 1}$
- **Minimum possible height:** for N nodes is $\mathbf{floor(\log_2 N)}$

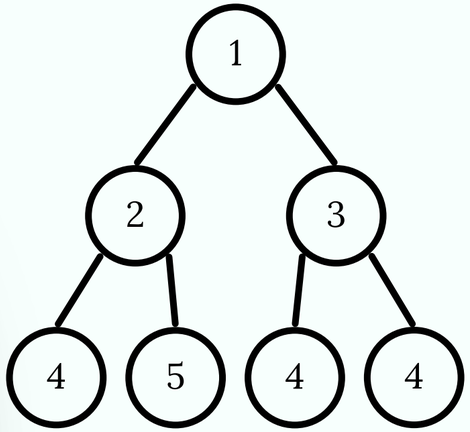
TYPES OF BINARY TREES



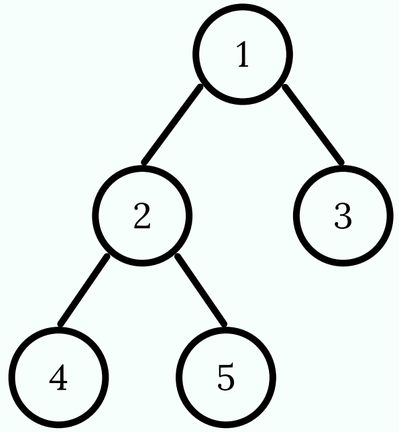
Full Binary Tree: Every non-leaf node has exactly two child nodes



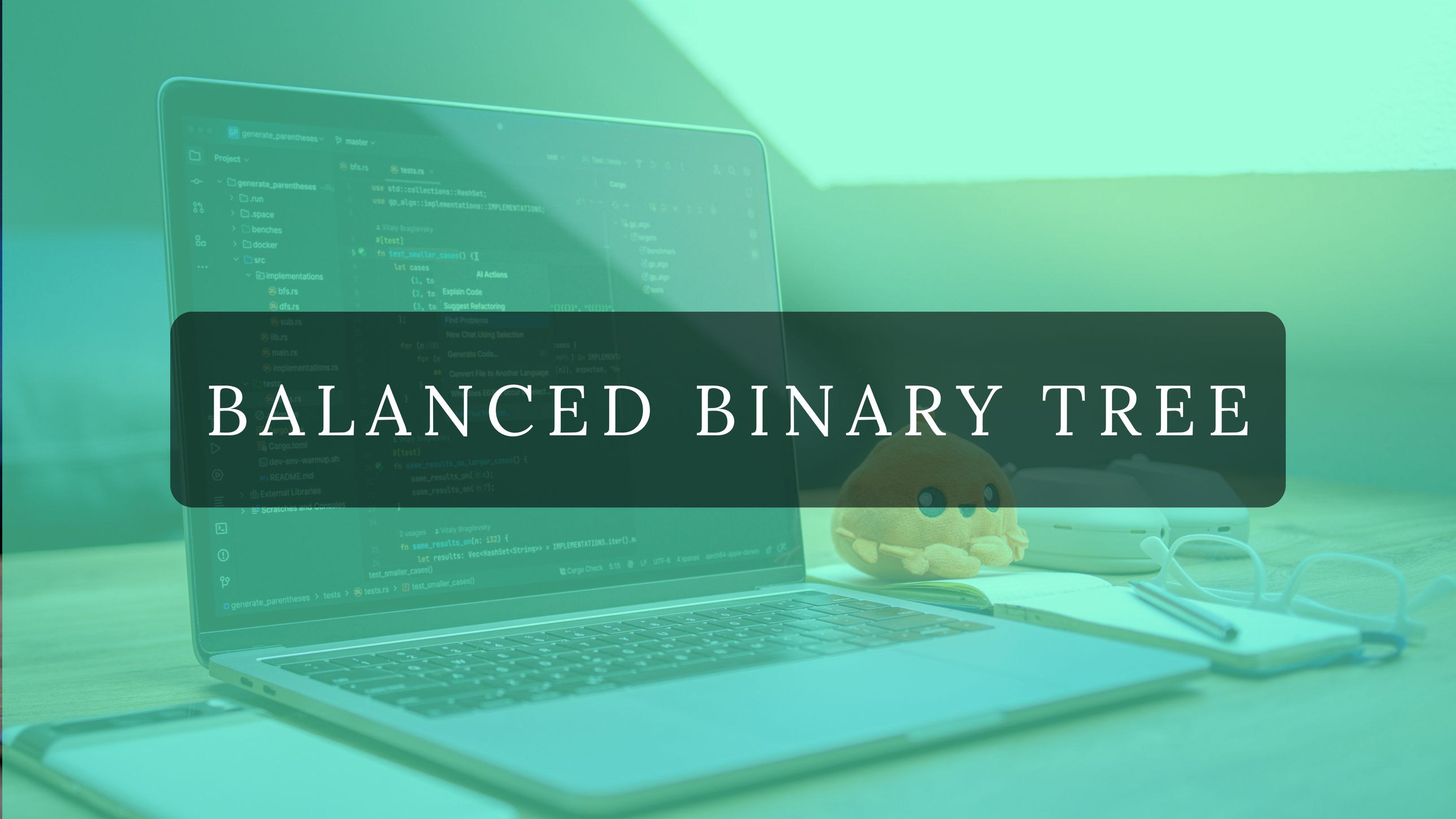
Complete Binary Tree: All levels are fully filled except possibly the last, which is filled from left to right.



Perfect Binary Tree: Every level is completely filled, and all leaves are at the same depth.

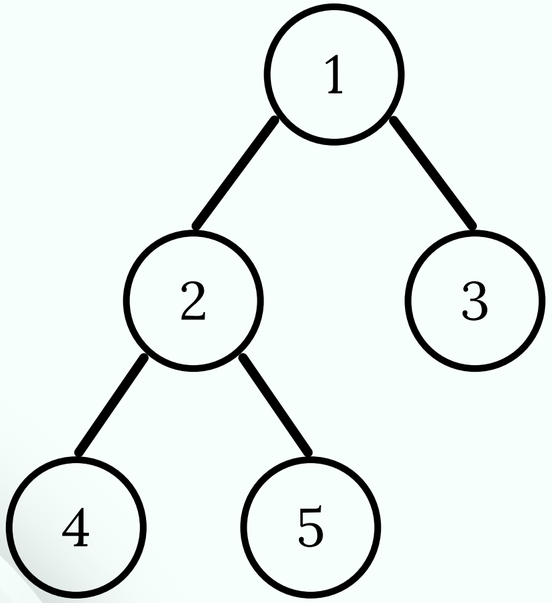


Balanced Binary Tree: The left and right subtrees under root node differ in height by at most 1.



BALANCED BINARY TREE

MINIMUM HEIGHT OF A BINARY TREE



If every level is as full as possible up to height **h**, the total number of nodes is:

$n \leqslant 1 + 2 + 4 + \ldots + 2^h = 2^{(h+1)} - 1$ (Geometric Series Formula)

$n \leqslant 2^{(h+1)} - 1$

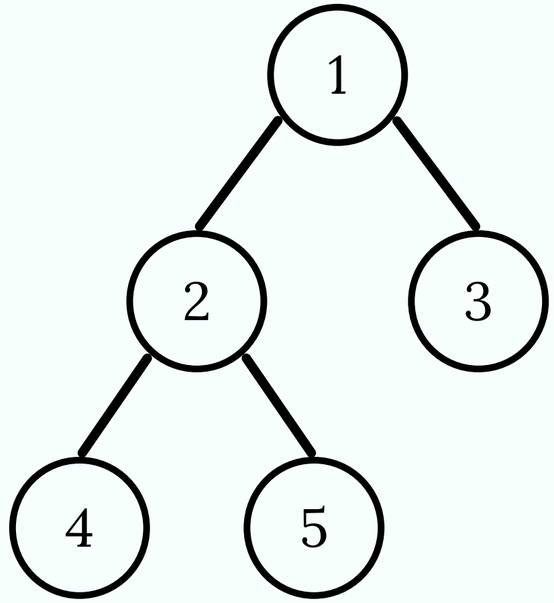
$n + 1 \leqslant 2^{(h+1)}$

$\log_2(n+1) \leqslant h + 1$

$\mathbf{h \geqslant \log_2(n+1) - 1}$

Therefore, the minimum possible height of a Binary Tree with **n** nodes: approximately $\mathbf{h \approx \log_2(n)}$.
The height of balanced binary trees stays close to this minimum

BALANCED BINARY TREE

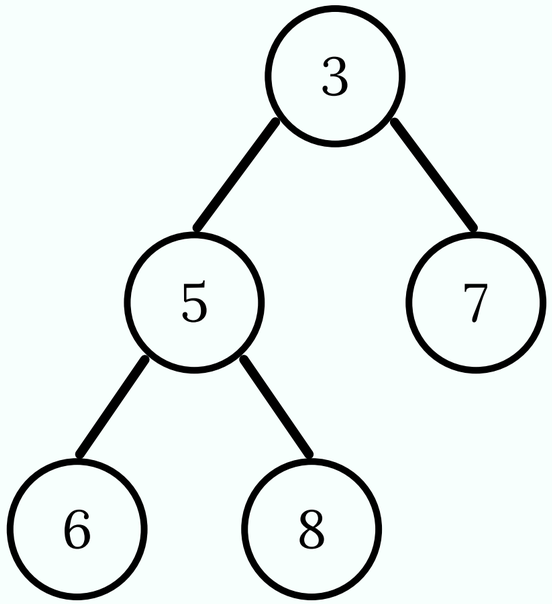


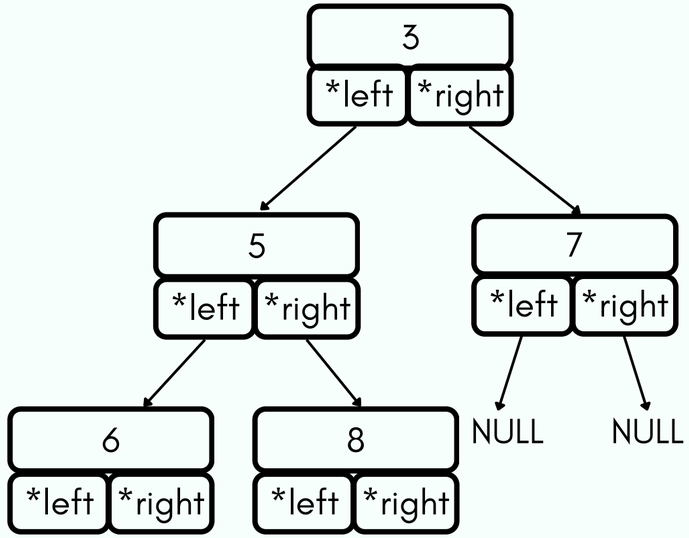
Balanced Binary Tree: The left and right subtrees under root node differ in height by at most 1
A balanced binary tree ensures that the **height** of a binary tree with n nodes is minimized to approximately $\mathbf{log_2(n)}$, instead of a maximum of **n**. This is important for ensuring the time complexity of Binary Search Tree operations (Search, Insertion and Deletion) are all **O(log n)**



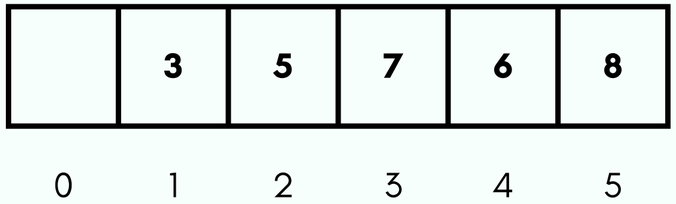
BINARY TREE REPRESENTATION

BINARY TREE REPRESENTATION





Linked Node Representation



Array Representation

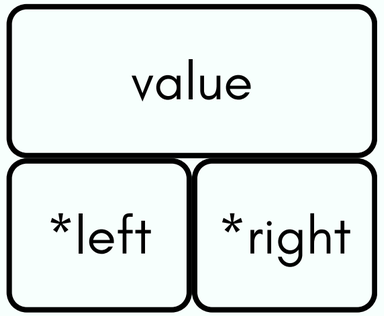
Binary Trees can be represented in two ways:

1. Linked Node Representation
2. Array Representation

BINARY TREE REPRESENTATION: LINKED NODE

```
class Node{
public:
    int value;
    Node* left;
    Node* right;

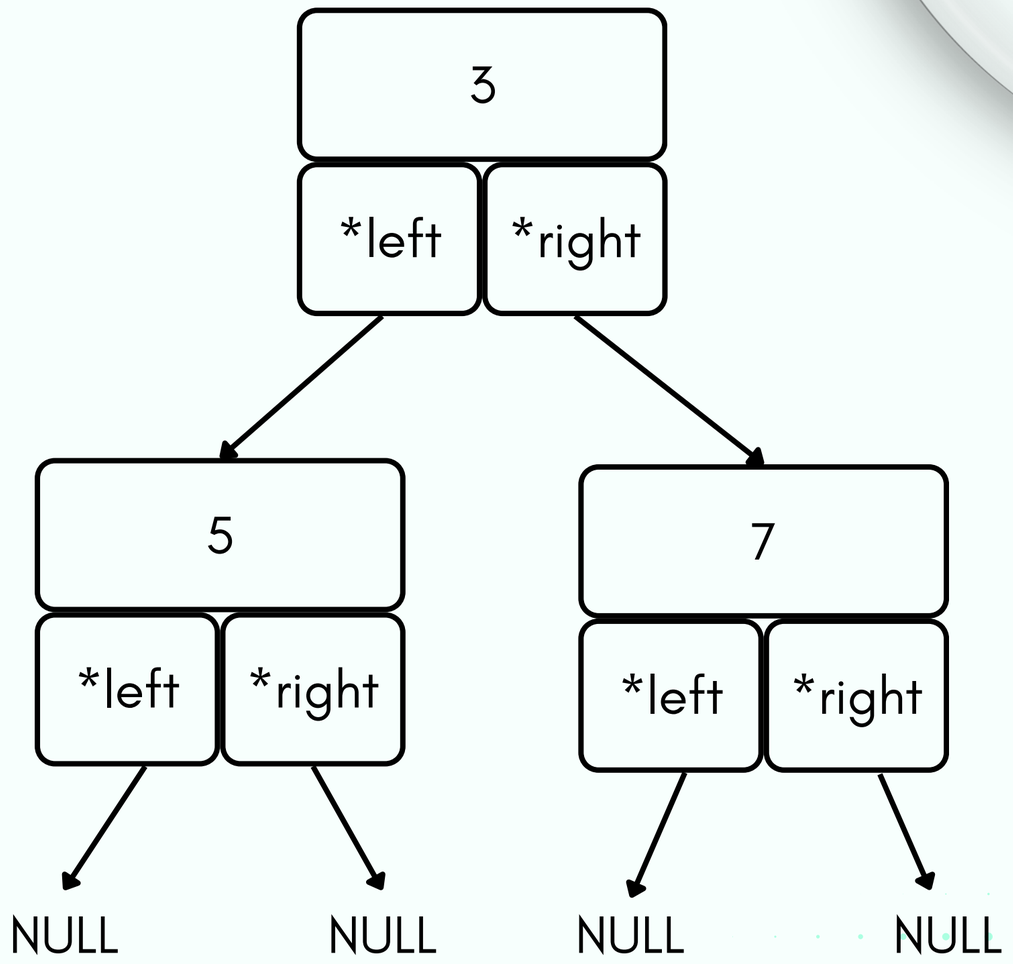
    Node(int x)
    {
        this->value = x;
        this->left = NULL;
        this->right = NULL;
    }
};
```



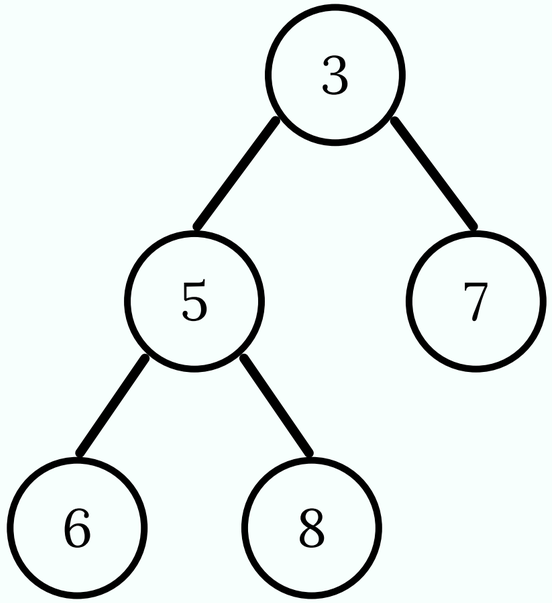
Each node will have three attributes: value, left pointer and right pointer

BINARY TREE REPRESENTATION: LINKED NODE

```
int main()
{
    Node* root = new Node(3);
    Node* node1 = new Node(5);
    Node* node2 = new Node(7);
    root->left = node1;
    root->right = node2;
}
```



BINARY TREE REPRESENTATION: ARRAY



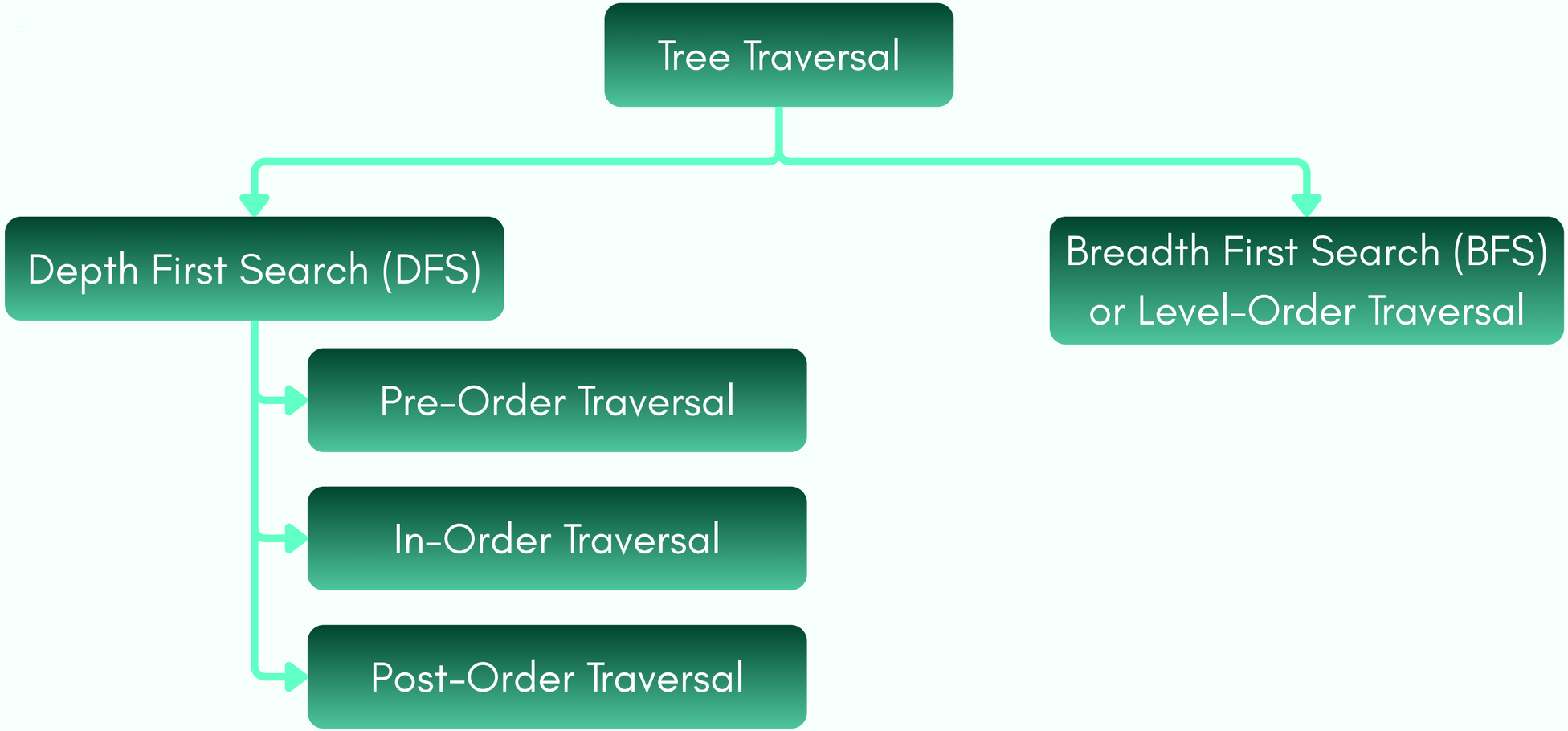
	3	5	7	6	8
0	1	2	3	4	5

- Root stored at index **1**
- For each node **i (i = 1 to number of nodes)**:
  - left child stored at index **2i**
  - right child stored at index **2i + 1**
  - parent stored at index **floor(i/2)**
- Index 0 is unused (For easier formula ↑)
- If a child does not exist, NULL or -1 is placed in the corresponding index



TREE TRAVERSAL TECHNIQUES

TREE TRAVERSAL TECHNIQUES

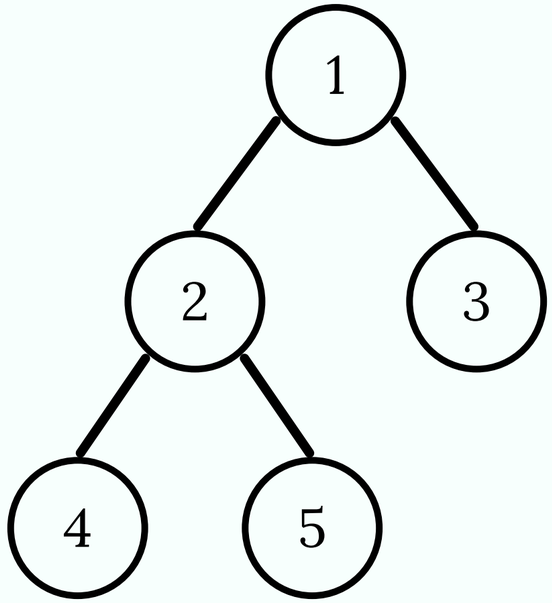


Note: All tree traversal techniques start at root node

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence:

Parent → Left → Right



PRE-ORDER TRAVERSAL

Code

```
void pre_order_traversal(Node* node)
{
    if(node == NULL){
        return;
    }
    cout << node->value << " ";
    pre_order_traversal(node->left);
    pre_order_traversal(node->right);
}
```

```
int main()
{
    Node* root = new Node(3);
    Node* node1 = new Node(5);
    Node* node2 = new Node(7);
    Node* node3 = new Node(4);
    root->left = node1;
    root->right = node2;
    node1->left = node3;

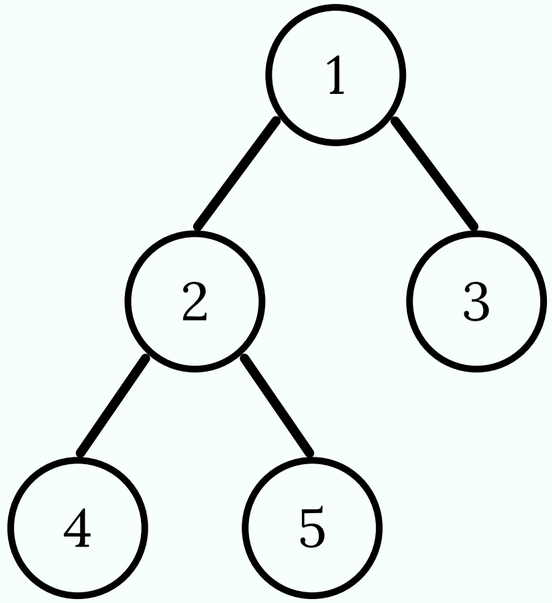
    pre_order_traversal(root);
}
```

- Visit parent first
- Visit left sub-tree recursively next
- Visit right sub-tree recursively last

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

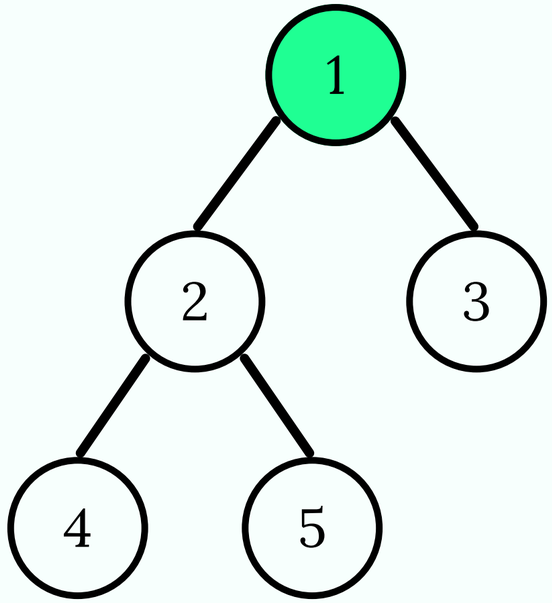


Pre-Order Traversal Sequence:

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	
3	
4	
5	
6	
7	
8	
9	
10	



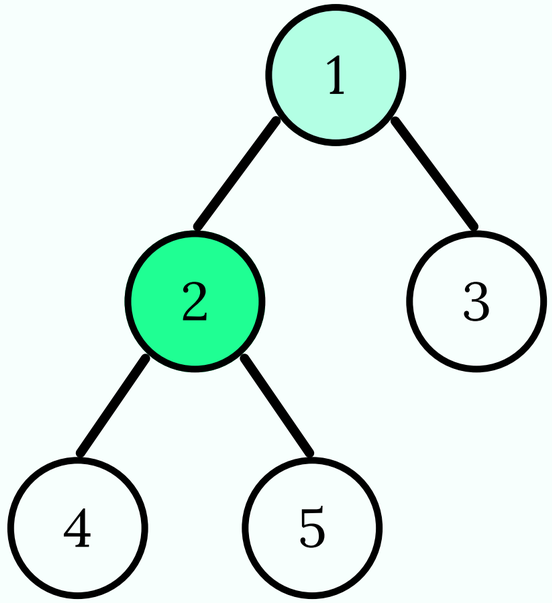
Pre-Order Traversal Sequence:

1

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	
4	
5	
6	
7	
8	
9	
10	



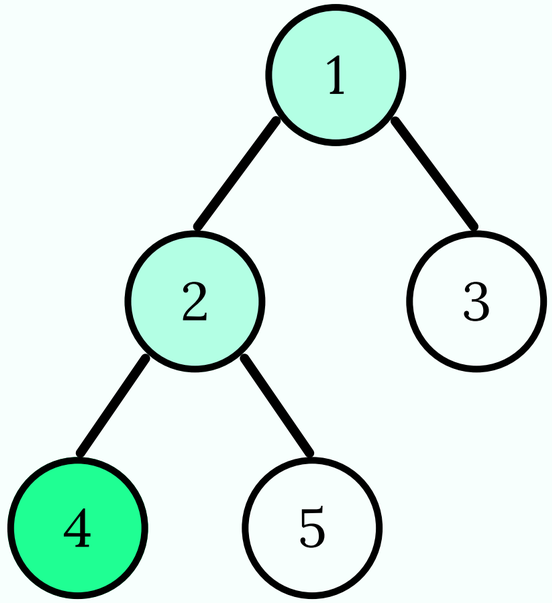
Pre-Order Traversal Sequence:

1 2

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	
5	
6	
7	
8	
9	
10	



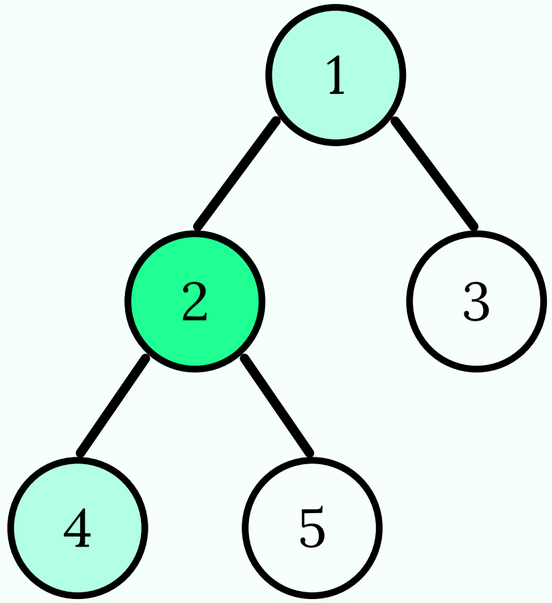
Pre-Order Traversal Sequence:

1 2 4

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	
6	
7	
8	
9	
10	



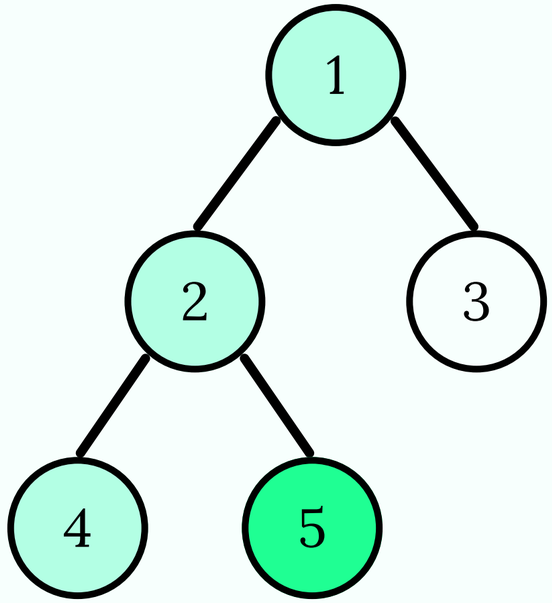
Pre-Order Traversal Sequence:

1 2 4

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	pre_order_traversal (Node(5))
6	
7	
8	
9	
10	



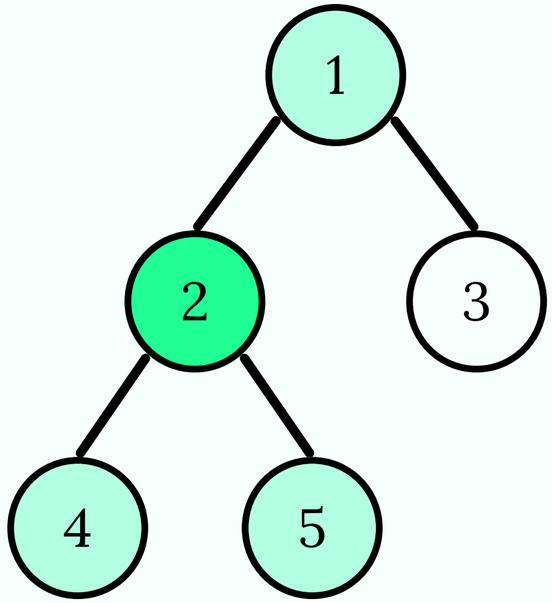
Pre-Order Traversal Sequence:

1 2 4 5

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	pre_order_traversal (Node(5))
6	return to pre_order_traversal (Node(2))
7	
8	
9	
10	



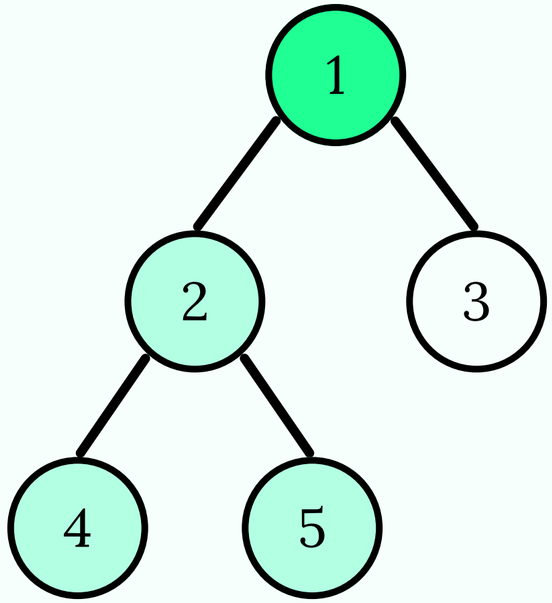
Pre-Order Traversal Sequence:

1 2 4 5

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	pre_order_traversal (Node(5))
6	return to pre_order_traversal (Node(2))
7	return to pre_order_traversal (Node(1))
8	
9	
10	



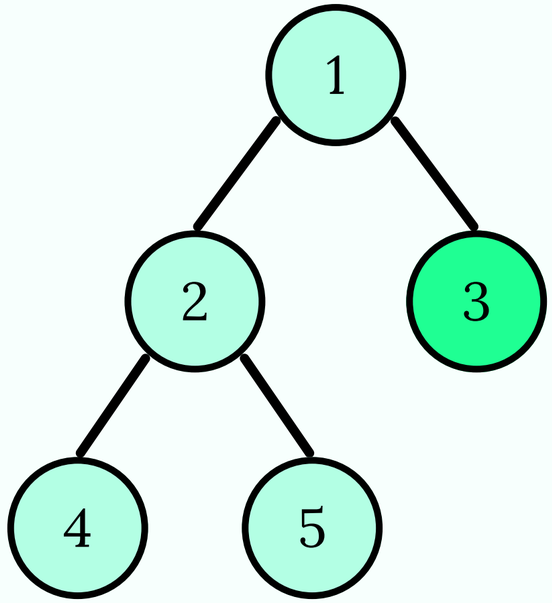
Pre-Order Traversal Sequence:

1 2 4 5

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	pre_order_traversal (Node(5))
6	return to pre_order_traversal (Node(2))
7	return to pre_order_traversal (Node(1))
8	pre_order_traversal (Node(3))
9	
10	



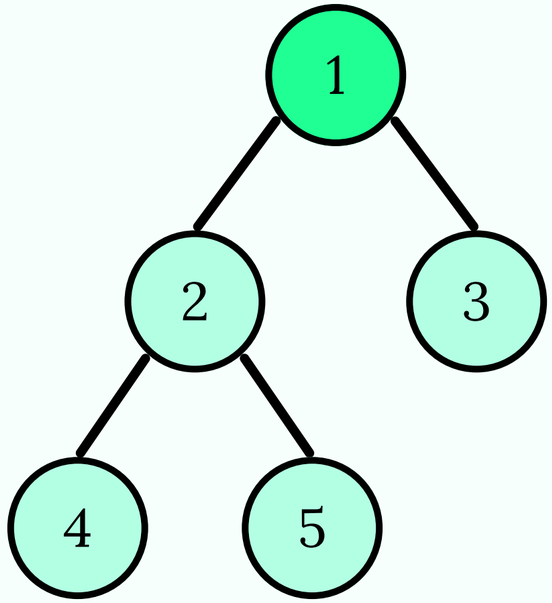
Pre-Order Traversal Sequence:

1 2 4 5 3

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	pre_order_traversal (Node(5))
6	return to pre_order_traversal (Node(2))
7	return to pre_order_traversal (Node(1))
8	pre_order_traversal (Node(3))
9	return to pre_order_traversal (Node(1))
10	



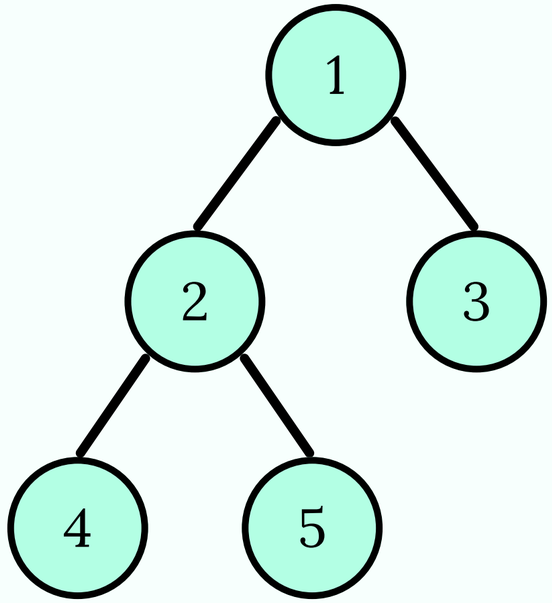
Pre-Order Traversal Sequence:

1 2 4 5 3

PRE-ORDER TRAVERSAL

Pre-Order Traversal visits nodes in the sequence: **Parent → Left → Right**

Step	Function Calls
1	pre_order_traversal (Node(1))
2	pre_order_traversal (Node(2))
3	pre_order_traversal (Node(4))
4	return to pre_order_traversal (Node(2))
5	pre_order_traversal (Node(5))
6	return to pre_order_traversal (Node(2))
7	return to pre_order_traversal (Node(1))
8	pre_order_traversal (Node(3))
9	return to pre_order_traversal (Node(1))
10	return to main()



Pre-Order Traversal Sequence:

1 2 4 5 3

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

